

חוברת הכנה למבחן: Large Language Models

גרסה מורחבת לפי התמלילים והמצגת המאוחדת

כולל נוסחאות, אלגוריתמים, דוגמאות, שאלות מרצה, דגשי מבחן, benchmarks, datasets ותרשימים.

איך להשתמש בחוברת

החוברת בנויה כמו חומר למידה למבחן ולא כמו תקציר. בכל נושא מופיעים הרעיון המרכזי, נוסחאות, אלגוריתם או פסאודו-קוד כשיש, דוגמאות, שאלות שהמרצה הצביע עליהן במצגת או בתמליל, ותשובות מורחבות. התמלילים הופקו מ-OpenWhisper ולכן תיקנתי מונחים לפי ההקשר: למשל "Bradly" תוקן ל-Bradley-Terry, "סוויגלו" ל-SwiGLU, "רופ" ל-RoPE, וכדומה.

דגש מבחן מתוך שיעור 12 והמצגת

המבחן הוא חומר פתוח, אך לא נועד לבדוק שינון. לפי דגשי המרצה: כל מה שהופיע בהרצאות, בשקפים או נאמר בעל פה הוא fair game; מקומות עם מילים כמו discuss / why / consider / think / why חשודים במיוחד; יש להכיר גם שמות datasets ומה הם מכילים; לא נדרשים מספרים מדויקים אבל כן סדרי גודל; יש להבין גרפים, דיאגרמות ונוסחאות. לכן בחוברת יש גם "איך לענות" ולא רק "מה ההגדרה".

What you should know

- Anything in lectures, either on slides or verbally, is fair game
 - places with "discuss", "consider", "think", "why" are especially sus
 - Including dataset names and what they contain
- I won't ask for exact numbers but you should know the rough scales
- You should understand graphs and diagrams. All of them.
 - Even (especially) weird looking ones.
 - Even if weren't discussed in details.
- I didn't use a lot of math notation. When I do, you should understand it.

דגשי המרצה: מה נחשב חומר למבחן ומה חשוב להבין (all_lectures.pdf, עמוד 1270)

Kinds of questions

- Definitions, what does the equation means, etc
- A model exhibits some behavior
 - what can be the source?
 - how would you fix?
 - which stages of training?
 - how will data look like?
 - how would you generate it?
 - how would you evaluate it?

סוגי שאלות צפויים: מקור התנהגות של מודל, איך לתקן, איך נראה הדאטה, איך להעריך (עמוד 1272)

מפת נושאי הקורס

אשכול	מה צריך לדעת	דוגמת שאלת מבחן
LLM systems	LPU, NTP, לולאה, מערכת סביב המודל, היסטוריית ציאט וכלים	למה LLM "לא באמת" מחפש באינטרנט אבל ChatGPT כן?
Transformer	Self-attention, causal mask, MHA, residual normalization, MLP	איך causal mask מונע דליפת מידע מהעתיד?
Modern Transformer	RMSNorm, SwiGLU/gating, RoPE, MoE GQA/MQA	מה משפר SwiGLU לעומת MLP רגיל ומה המחיר?
/Pretraining/Data Eval	NTP, CE, perplexity/BPB, tokenization, data filtering, contamination, scaling laws	למה PPL תלוי ב-tokenizer ולמה benchmark contamination מסוכן?
Post-training	SFT, preference data, Bradley-Terry reward model, PPO/RLHF, DPO, RLAIIF, RLVR	למה DPO נשאר קרוב יותר למודל הייחוס מאשר RLHF?
/Reasoning/code tools	ICL, CoT, biases, test-time compute, code data, agents/tool calling	מתי RLVR יעבוד ומתי לא?
Inference efficiency	-latency/throughput, prefill/decode, KV cache, batching, quantization, speculative decoding	למה decode memory-bound ו-prefill compute-bound?

אשכול	מה צריך לדעת	דוגמת שאלת מבחן
VLM	CLIP/SigLIP, ViT, LLaVA-style adapter, VQA datasets, multimodal post-training	למה question-only baseline ב-VQA יכול להצליח?
Interpretability	,logit lens, probing, SAE, interventions activation patching	מה activation patching בודק ולמה זה חזק יותר מ-correlation בלבד?

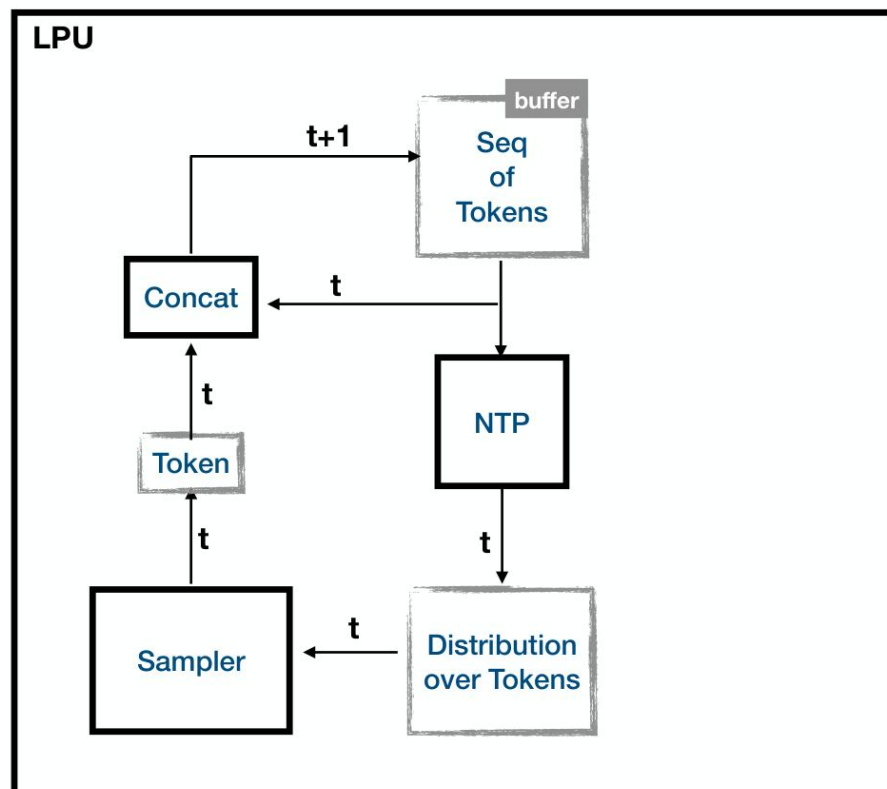
1. מבוא: LLM, NTP, LPU ומערכת LLM

הגדרת הבסיס בקורס: מודל שפה הוא פונקציה שמקבלת רצף טוקנים ומחזירה התפלגות הסתברות על הטוקן הבא. כל היכולות המורכבות שאנו רואים - שיחה, שימוש בכלים, חיפוש, כתיבת קוד - נבנות על גבי פונקציה זו באמצעות לולאות, פורמטים, post-training ומערכת סביב המודל.

$$F_{\theta}(x_1, \dots, x_t) = P_{\theta}(x_{t+1} \mid x_1, \dots, x_t)$$

$$P_{\theta}(x_{1:T}) = \prod_{t=1}^T P_{\theta}(x_t \mid x_{<t})$$

LLM



לולאת יצירה: התפלגות על טוקנים → concat → token → sampler → קונטקסט חדש (עמוד 20)

Large Language Models

LLMs cannot:

- chat!
- search the web!!
- use tools!!!!

LLMs are **single-step*** machines.
The horizon is **multi-step**.



הטענה המרכזית: LLM הוא חד-צעדי, אך האופק של מערכת LLM הוא רב-צעדי (עמוד 35)

The Power of Loops

- An LPU is an NTP wrapped in a loop.
- A ChatBot is an LPU wrapped in a loop.
- An Agent is an LPU wrapped in a loop.

כוחן של לולאות: LPU, chatbot ו-agent הם עטיפות שונות סביב אותה יחידת עיבוד (עמוד 37)

שאלה / נקודת מחשבה: האם LLM יכול לשוחח, לחפש באינטרנט או להשתמש בכלים?

תשובה מורחבת: ליבת המודל, כ-NTP, לא "מחפשת" ולא "משתמשת" בכלי. היא מחזירה טוקן או טקסט. מערכת LLM יכולה להגדיר פורמט שבו הטקסט של המודל מתפרש כבקשת כלי, להריץ את הכלי, להחזיר את התוצאה לתוך הקונטקסט, ואז לקרוא שוב למודל. לכן היכולת שייכת למערכת, לא רק למודל.

דוגמה: בפרומפט המערכת כותבת: אם צריך מזג אוויר, החזר JSON עם שם כלי וארגומנטים. המודל פולט בקשת כלי מובנית, הקונטרולר מריץ API ומכניס את התוצאה לשיחה.

שאלה / נקודת מחשבה: למה חשוב להבחין בין LLM לבין LLM system?

תשובה מורחבת: במבחן עלולה להופיע התנהגות של מוצר - למשל זיכרון, חיפוש, ניהול היסטוריה או כלי - והשאלה תהיה מאיזה שלב היא מגיעה. תשובה טובה תפריד בין יכולת שנלמדה במשקולות, פורמט שנלמד ב-post-training, ולוגיקה מערכתית חיצונית.

דוגמה: אם מודל עונה "חיפשתי ומצאתי", זה לא מוכיח שהמודל עצמו חיפש. ייתכן שהמערכת נתנה לו תוצאת חיפוש כטקסט.

נוסחאות ואלגוריתם בסיסי

$$L_{LM}(\theta) = - \sum_t \log P_{\theta}(x_t | x_{<t})$$

יצירת טקסט אוטוגרסיבי

```
context = tokenize(prompt)
for step in 1..max_new_tokens:
    logits = model(context)
    probs = softmax(logits[-1])
    next = sample_or_argmax(probs)
    context.append(next)
    if next == EOS: break
return detokenize(context)
```

דוגמה קצרה

הפרומפט "בירת צרפת היא" גורם ל-NTP לתת הסתברות גבוהה לטוקן "פריז". ה-LPU מוסיף את הטוקן לקונטקסט וממשיך. מערכת צ'אט תעטוף את זה בהודעות /system/user assistant ותשמור היסטוריה.

Self-Attention-1 Transformer .2

ה-Transformer הוא ארכיטקטורת הרצף המרכזית במודלי שפה. היכולת הקריטית היא self-attention: כל טוקן בונה ייצוג חדש על ידי שקלול ייצוגים של טוקנים אחרים. במודל שפה causal, כל מיקום רשאי להסתכל רק על עברו ועל עצמו.

How to combine vectors?

- Sum (including weighted sum)
- Max
- Concatenate
- Hybrid $(\mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3 + \mathbf{v}_4) \oplus \mathbf{v}_5 \oplus \mathbf{v}_6$
- Recursive (RNN, LSTM, GRU, ...) ← iykyk
(if you know you know)
- **Attention / Self-attention**

תחילת דיון attention במצגת (עמוד 114)

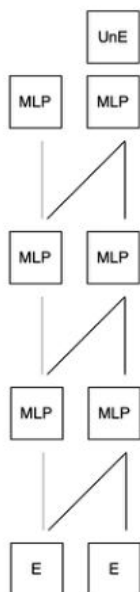
Attention-based NTP - high level idea

$$\begin{aligned}
 \text{NTP}(\mathbf{x}_1, \dots, \mathbf{x}_n) &= \text{MLP}(\text{combine}(\mathbf{x}_1, \dots, \mathbf{x}_n)) \\
 &= \text{MLP}(\text{Att}(\underbrace{f^1(\mathbf{x}_n)}_{\mathbf{q}}, \underbrace{f^2(\mathbf{x}_1), \dots, f^2(\mathbf{x}_n)}_{\mathbf{K}}, \underbrace{f^3(\mathbf{x}_1), \dots, f^3(\mathbf{x}_n)}_{\mathbf{V}}))
 \end{aligned}$$

Why do we need **different transformations** of $\mathbf{x}$ for each of $\mathbf{q}$, $\mathbf{K}$, $\mathbf{V}$?

Causal mask: מניעת גישה לטוקנים עתידיים (עמוד 128)

Transformer Decoder LM



Transformer block: attention, residual, normalization (עמוד 144)

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}((QK^T / \sqrt{d_k}) + M) V$$

$$M_{ij} = 0 \text{ if } j \leq i, \text{ else } -\infty \quad (\text{causal mask})$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_0$$

שאלה: למה מחלקים ב- $\sqrt{d_k}$?

תשובה: מכפלה פנימית של וקטורים מממד גבוה מקבלת שונות גדולה יותר ככל שהממד גדל. בלי החלוקה הלוגיטים ב-softmax נעשים גדולים מדי, softmax נהיה חד מדי, והגרדיאנטים פחות יציבים.

דוגמה: אם $d_k=1024$, מכפלות QK יכולות להיות גדולות בהרבה מאשר ב- $d_k=64$. החלוקה שומרת את הסקאלה דומה.

שאלה: למה צריך causal mask?

תשובה: באימון next-token prediction, אם הטוקן במיקום t רואה טוקנים עתידיים, המשימה הופכת לרמאות: המודל יכול להעתיק מידע מהתשובה. המסכה מכריחה אותו להשתמש רק בקונטקסט הקודם.

דוגמה: במשפט "אני גר ב___", בעת חיזוי הטוקן "תל", אסור למודל לראות את המילה הבאה "אביב".

3. Transformer מודרני: RMSNorm, SwiGLU, RoPE, MoE

מודלים מודרניים כמו משפחות LLaMA/Qwen/DeepSeek משתמשים בוריאנטים יעילים ויציבים יותר מה-Transformer המקורי. ארבעה רכיבים בולטים בקורס: RMSNorm, SwiGLU, RoPE ו-MoE-1.

Last time

- The basic Transformer architecture
 - (causal) Self-attention
 - Multi-head attention
 - MLP
 - Residual connections
 - Position-embeddings

המעבר במצגת מ-Transformer בסיסי ל-Modern Transformers (עמוד 181)

RMSNorm 3.1

RMSNorm הוא נרמול לפי גודל הווקטור במקום לפי ממוצע ושונות כמו LayerNorm. הוא מסיר חלק מהחישוב ועדיין שומר על יציבות סקאלה.

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}$$

$$\text{RMSNorm}(x) = g \odot x / \text{RMS}(x)$$

שאלה: מה RMSNorm נותן?

תשובה: הוא זול יותר מ-LayerNorm, פשוט יותר, ונפוץ ב-LLMs מודרניים. המטרה היא למנוע מהאקטיבציות "לברוח" בסקאלה לאורך שכבות רבות.

דוגמה: אם $x=[3,4]$, אז $RMS=\sqrt{(9+16)/2}=3.535$. הווקטור מנורמל בערך ל-[0.85,1.13] ואז מוכפל בפרמטר scale נלמד.

3.2 Swish, GLU, ו-SwiGLU - מה זה gated?

זה היה חסר בסיכום הקודם ולכן כאן יש פירוט. MLP רגיל ב-Transformer מפעיל טרנספורמציה ליניארית, אקטיבציה, ואז ליניארית חזרה. ב-GLU מוסיפים שער: ערוץ אחד מייצר "תוכן", וערוץ שני מייצר "שער" שקובע כמה מהתוכן יעבור. SwiGLU משתמש ב-Swish כשער.

New activation: SwiGLU

שקפי SwiGLU: החלפת MLP רגיל ב-gated MLP (עמוד 190)

$$\text{Swish}(x) = x \cdot \text{sigmoid}(x)$$

$$\text{GLU}(x) = (xW_v) \odot \sigma(xW_g)$$

$$\text{SwiGLU}(x) = (xW_v) \odot \text{Swish}(xW_g)$$

$$\text{FFN_SwiGLU}(x) = W_o [(xW_v) \odot \text{Swish}(xW_g)]$$

שאלה: מה פירוש gated?

תשובה: שער הוא מנגנון כפל איבר-איבר. אם רכיב השער קרוב ל-0, המידע בערוץ התוכן נחסם; אם הוא גבוה, המידע עובר. זה מאפשר ל-MLP לבחור דינמית אילו תכונות להפעיל לפי ההקשר.

דוגמה: במשימת תרגום, רכיבים הקשורים למגדר יכולים להיפתח רק כשהקונטקסט דורש התאמת זכר/נקבה.

שאלה: למה לא פשוט ReLU/GELU?

תשובה: SwiGLU נותן אינטראקציה מולטיפליקטיבית בין שני מסלולים. מבחינה אינטואיטיבית, המודל לא רק מחשב feature אלא גם מחשב האם feature זה רלוונטי עכשיו. המחיר הוא עוד פרמטרים/חישוב בשכבת ה-FFN, אם לא מקטינים ממדים בהתאם.

דוגמה: ב-FFN רגיל: $h = \text{GELU}(xW_1)$. ב-SwiGLU: $h = (xW_v) \odot \text{Swish}(xW_g)$, ולכן אותה שכבה יכולה לבצע "תוכן כפול תנאי".

RoPE - Rotary Positional Embeddings 3.3

Attention כשלעצמו אינו יודע סדר. צריך קידוד מיקום. RoPE מכניס מידע מיקום על ידי סיבוב זוגות רכיבים בווקטורי Q ו-K בזווית שתלויה במיקום. היתרון: המוצר הפנימי בין Q ל-K תלוי בהפרש המיקומים, ולכן מתאים למרחקים יחסיים.

RoPE: Relative Position Encoding

RoPE: קידוד מיקום על ידי סיבוב וקטורים (עמוד 205)

$$q_m = R_m W_Q x_m, \quad k_n = R_n W_K x_n$$

$$\langle q_m, k_n \rangle \text{ depends on } (m - n)$$

שאלה: למה RoPE חשוב ב-long context?

תשובה: כי הוא מאפשר ל-attention ללמוד תלויות יחסיות בין מיקומים. עם זאת, הרחבת context מעבר למה שנראה באימון עדיין אינה טריוויאלית ודורשת התאמות או fine-tuning.

דוגמה: אם טוקן צריך להתייחס לסוגר שנפתח 200 טוקנים אחורה, המודל צריך לדעת לא רק "מיקום אבסולוטי" אלא מרחק יחסי.

MoE - Mixture of Experts 3.4

ב-MoE מחליפים לרוב את ה-FFN במספר מומחים. עבור כל טוקן, router בוחר מעט מומחים פעילים. כך מספר הפרמטרים הכולל גדל מאוד, אבל החישוב לכל טוקן גדל פחות.

Parameter Efficient MLP: MoE

MoE: רעיון כללי של experts ו-router (עמוד 228)

$\text{router}(x) \rightarrow \text{top-k experts}$

$$y = \sum_{e \in \text{TopK}(x)} \text{gate}_e(x) \cdot \text{Expert}_e(x)$$

שאלה: מה MoE משפר ומה המחיר?

תשובה: MoE מאפשר capacity גדול יותר בלי להפעיל את כל הפרמטרים לכל טוקן. המחיר: עומס לא מאוזן בין מומחים, תקשורת בין GPUs, routing לא יציב, צורך ב-load-balancing losses, ואינפרנס מורכב יותר.

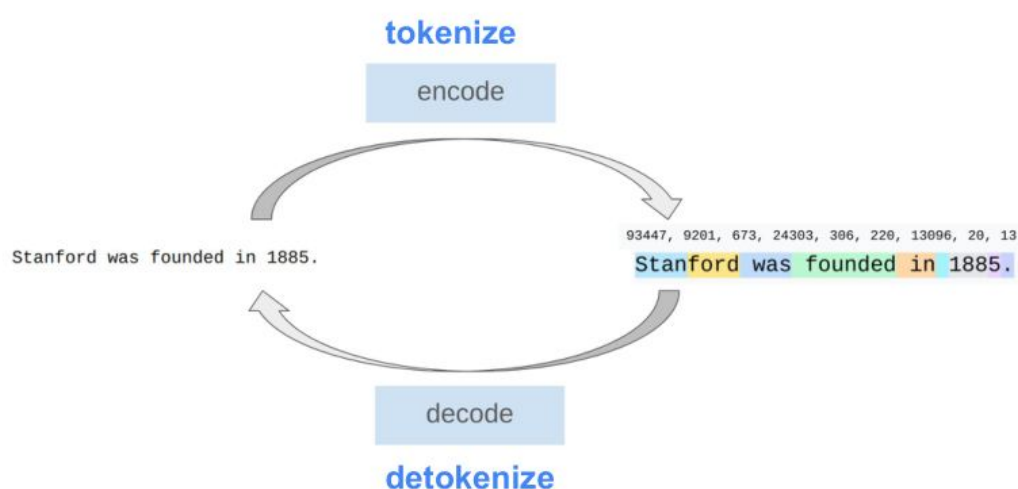
דוגמה: מודל עם 8 מומחים ו-top-2 מפעיל רק שני FFN לכל טוקן, אך עדיין מחזיק זיכרון לפרמטרים של כל שמונת המומחים.

Tokenization, Pretraining, Data .4

Constrained Decoding-1

השלב הראשון של LLM הוא pretraining על כמויות טקסט עצומות עם objective של next token prediction. לפני זה צריך tokenizer: מיפוי בין מחרוזות לבין רצף מזהי טוקנים.

Tokenization



Tokenizers convert between strings and sequences of integers (tokens)

Tokenizer: tokenize/detokenize בין טקסט לרצף טוקנים (עמוד 307)

Intrinsic quality metrics

$$-\sum_{i=1}^N \log_2 q(x_i | x_{1:i-1})$$

cross entropy

$$-\frac{1}{N} \sum_{i=1}^N \log_2 q(x_i | x_{1:i-1})$$

avg cross entropy

$$2^{-\frac{1}{N} \sum_{i=1}^N \log_2 q(x_i | x_{1:i-1})}$$

Perplexity

Cross entropy ו-perplexity כמדדי איכות פנימיים (עמוד 319)

$$L_{CE} = - \sum_t \log P_{\theta}(x_t | x_{<t})$$

$$PPL = \exp(\text{average cross entropy})$$

$$BPB = \text{average cross entropy} / \text{average bytes per token}$$

שאלה: מהו token טוב?

תשובה: טוקן הוא יחידת החיזוי. טוקניזציה טובה מאזנת בין אורך רצף קצר לבין יכולת לייצג שפות, קוד, מספרים וסימנים. אוצר מילים גדול מקצר רצפים אך מגדיל softmax ומייצר טוקנים נדירים; אוצר קטן מייצר רצפים ארוכים יותר.

דוגמה: באנגלית "unbelievable" עשוי להתפצל ל-un + believable; בעברית מילים עם תחיליות/סיומות עלולות להתפצל אחרת ולהקשות על מודל רב-לשוני.

שאלה: למה perplexity תלוי ב-vocabulary?

תשובה: כי PPL מחושב פר טוקן. אם tokenizer אחד מפצל משפט ל-10 טוקנים ואחר ל-20, ההשוואה הישירה אינה הוגנת. לכן BPB/bits-per-byte מתאים יותר להשוואה בין tokenizers.

דוגמה: מודל שמנבא 10 טוקנים ארוכים ומודל שמנבא 20 טוקנים קצרים יכולים לקבל PPL שונה למרות איכות דומה ברמת התווים.

BPE בקצרה

Byte Pair Encoding - סקיצה

```
vocab = all characters or bytes
while len(vocab) < budget:
    count all adjacent token pairs in corpus
    merge the most frequent pair into a new token
    add it to vocab
return vocab and merge rules
```

שאלה: מה tradeoff בטוקניזציה של מספרים?

תשובה: אם כל מספר שלם הוא טוקן נפרד, אי אפשר לכסות את כל המספרים. אם כל ספרה היא טוקן, רצפים ארוכים יותר אבל המודל יכול ללמוד חוקים ספרתיים. בפועל יש tokenizers שמפצלים מספרים בדרכים שיכולות להשפיע על arithmetic reasoning.

דוגמה: המספר 123456 יכול להיות token אחד, שלושה tokens כמו 123/456, או שש ספרות. כל בחירה משנה את דפוס הלמידה.

Constrained decoding

לפעמים רוצים שהמודל יפיק רק פורמט חוקי: JSON, קוד, תשובה מתוך רשימה, או טוקנים בשפה מסוימת. אז מסננים טוקנים אסורים בכל צעד ומנרמלים מחדש את ההסתברויות.

$$P_{\text{allowed}}(t) = \text{softmax}(\text{logits}_t + \text{mask}_{\text{allowed}})$$

שאלה: softmax לפני או אחרי סינון?

תשובה: אם מסננים logits ואז עושים softmax, ההסתברויות מנורמלות רק על הטוקנים המותרים. אם עושים softmax ואז מאפסים, צריך לנרמל מחדש. בפועל שני ניסוחים שקולים אם מנרמלים מחדש, אך עבודה על logits יציבה יותר.

דוגמה: אם רק A ו-B מותרים והמודל נתן גם C הסתברות גבוהה, אחרי סינון מסתכלים על היחס בין A ל-B בלבד.

benchmark-1 Data pipelines, filtering .5 contamination

הקורס שם דגש שהדאטה אינו רק "עוד קובץ". איכות, סינון, דדופליקציה והרכב הדאטה משפיעים על יכולות המודל, על bias, ועל תקפות ההערכה.

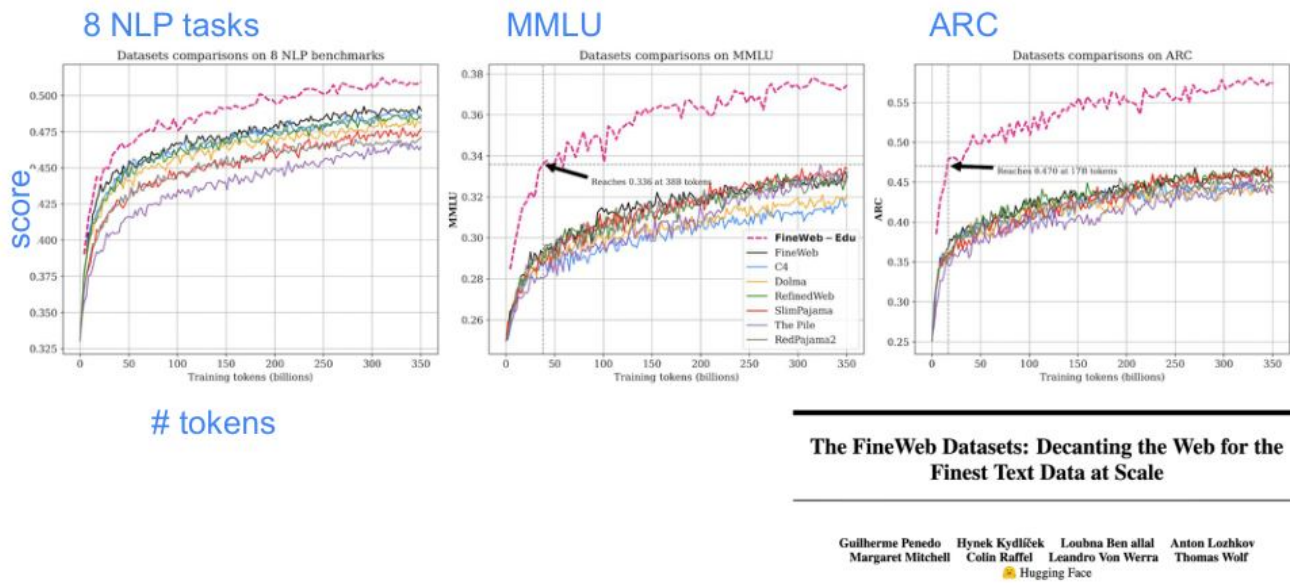
Another important issue to consider:

Benchmark contamination.

How do we know that we don't train on our benchmarks?

Benchmark contamination: האם אימנו על הדוגמאות שעליהן אנו בודקים? (עמוד 438)

High quality data may work better (FineWeb-EDU)



FineWeb-Edu: איכות דאטה מול ביצועים בבנצ'מרקים (עמוד 451)

שאלה: למה deduplication?

תשובה: אם אותה דוגמה מופיעה פעמים רבות, המודל עלול לשנן אותה, לתת לה משקל יתר, ולזהם הערכה. דדופליקציה יכולה להיות exact, fuzzy או containment. בסקייל גדול משתמשים ברעיונות כמו MinHash, Bloom filters או suffix arrays.

דוגמה: אם שאלת benchmark מופיעה בפורום ובדאטה האימוני 200 פעמים, הצלחה עליה לא מוכיחה הכללה.

שאלה: האם לאמן על דאטה רעיל?

תשובה: אין תשובה חד-משמעית. אם מסירים הכול, המודל אולי לא יכיר שפה רעילה ולא ידע לזהות/לסרב. אם משאירים הכול, המודל עלול ללמוד להפיק תוכן רעיל. לרוב מפרידים בין pretraining, סינון, safety data ו-post-training.

דוגמה: למודל בטיחות צריך לזהות קללות/איומים כדי לסרב או לנטר, אך לא בהכרח ללמוד לחקות אותם.

שאלה: מה benchmark contamination ומה עושים נגדו?

תשובה: זיהום benchmark קורה כאשר דוגמאות ההערכה או תשובותיהן נמצאות בדאטה האימון. זה מעלה ציונים באופן מלאכותי. אפשר לבדוק חפיפה טקסטואלית, להסיר דוגמאות דומות, להשתמש בבנצימרים חדשים/סודיים, או לבנות הערכה עם וריאציות.

דוגמה: אם HumanEval task נמצא בגיטהאב עם פתרון, מודל קוד שאומן על גיטהאב עלול "להכיר" אותו.

סוגי סינון דאטה

סוג סינון	מה עושים	סיכון/שאלה למבחן
Language ID	מזהים שפה ושומרים/מסירים לפי יעד האימון	מה קורה לשפות low-resource אם מסננים אגרסיבית?
Quality filters	חוקים, perplexity, או classifier לאיכות	מי קובע מהו "איכותי"? האם מסירים סלנג/דיאלקטים?
Toxic/harmful filters	מסירים או מסמנים תוכן פוגעני/מסוכן	האם המודל צריך להכיר תוכן כזה כדי להימנע ממנו?
PII filters	regex/classifier לפרטים אישיים	כשלי privacy ושינון מידע פרטי
Deduplication	exact/fuzzy/substring	שמירת diversity מול הסרת זיהום ושינון
Model-based filtering	LLM/embedding model מדרג איכות למשל FineWeb-Edu	האם classifier מכניס bias של המודל החזק?

.6 Evaluation: intrinsic metrics

datasets-1 extrinsic benchmarks

המרצה הדגיש במיוחד שמות datasets ומה הם מכילים. לכן זהו אחד החלקים החשובים ביותר למבחן. צריך לדעת לא רק את השם אלא גם איזו יכולת הוא בודק ומה המלכודות.

(some) Extrinsic eval QA datasets

- **MMLU** - Factual knowledge
- **HellaSwag** - Commonsense situational knowledge
- **WinoGrande** - Low level common sense
- **ARC** - Questions that require scientific understanding and logical reasoning
- **GPQA** - Graduate-Level Google-Proof Questions
- **OpenBookQA** - Questions that require reasoning, supporting facts supplied with the question.
- **BoolQ** - Hard Yes/No questions from Google query logs
- **PIQA** - Physical Common Sense

רשימת QA benchmarks מהמצגת: MMLU, HellaSwag, WinoGrande, ARC, GPQA, OpenBookQA, BoolQ, PIQA (עמוד 328)

Intrinsic vs Extrinsic

סוג מדד	דוגמה	יתרון	חסרון
Intrinsic	Perplexity, cross entropy BPB	זול, אוטומטי, קשור לאימון	לא מודד alignment, reasoning, tool use או בטיחות
Extrinsic	MMLU, ARC, GSM8K, HumanEval, VQA	מודד משימה שמעניינת אותנו	רגיש לפורמט, contamination, בחירת prompt, grading
Human eval	השוואת תשובות על ידי בני אדם	מודד איכות אמיתית יותר	יקר, איטי, subjective, קשה לשחזור

Benchmarks ודאטהסטים שהוזכרו - מה הם בודקים

שם	מה מכיל	יכולת מרכזית	מלכודת/דגש
MMLU	שאלות בחירה מרובה בתחומי ידע רבים	ידע עובדתי ורוחב תחומים	יכול להיות מושפע מ-contamination; לא מספיק לבדוק "חשיבה"
HellaSwag	השלמת תרחישים יומיומיים/ commonsense	ידע מצבי והיגיון יומיומי	מודלים יכולים לנצל artifacts של ניסוח
WinoGrande	Winograd-style coreference	commonsense נמוך-רמה והבנת ייחוס	שינוי מילה קטנה יכול להפוך תשובה
ARC	שאלות מדעיות והיגיון מדעי	הבנה מדעית ולוגית	ARC-Challenge קשה יותר; לא רק שליפת ידע
GPQA	שאלות graduate-level "Google-proof"	ידע מומחה והיסק	גם אנשים לא מומחים מתקשים; grading קשיח
OpenBookQA	שאלות עם עובדות תומכות מסופקות	reasoning עם facts נתונים	בודק שימוש בעובדות, לא רק ידע חיצוני
BoolQ	שאלות yes/no מלוגים של Google	הבנת שאלה ופסקה	פורמט כן/לא רגיש לניסוח
PIQA	Physical commonsense	היגיון פיזיקלי יומיומי	תשובות לעיתים נראות דומות; דורש ידע על אובייקטים
GSM8K	בעיות מילוליות בחשבון	multi-step arithmetic reasoning	CoT/RLVR יכול לשפר מאוד; חשוב verifier לתשובה
/ HumanEval MBPP	בעיות תכנות עם unit tests	סינתזת קוד מפונקציה/הוראה	pass@k תלוי במספר ניסיונות; tests אינם מוכיחים נכונות מלאה
VQA	שאלות על תמונה עם תשובות אנושיות	ראייה + שפה	question-only baseline יכול להצליח בגלל bias בדאטה
ScienceQA	שאלות מדעיות לעיתים מולטימודליות	ידע מדעי + היסק	צריך לדעת האם נדרש image או די בטקסט
Conceptual Captions 3M	זוגות image-caption	אימון שלב alignment ל-VLM	קפציות קצרות לא מספיקות ל-dialog עשיר
/ FineWeb FineWeb-Edu	קורפוס web מסונן; Edu מדורג לחומר חינוכי	pretraining איכותי וידע	classifier-based filtering יכול להעדיף סגנון מסוים

שאלה: איך לענות לשאלה "איזה benchmark מתאים?"

תשובה: צריך לזהות את היכולת שרוצים לבדוק: ידע factual, commonsense, חשבון רב-שלבי, תכנות, שימוש בכלי, ראייה+שפה. אחר כך לבחור benchmark שמבודד יחסית את היכולת ולציין מגבלות.

דוגמה: אם רוצים לבדוק האם VLM באמת משתמש בתמונה, בוחרים VQA אבל מוסיפים question-only ו-image-only baselines כדי לגלות bias.

שאלה: למה פורמט ה-benchmark חשוב?

תשובה: מודלי base לא בהכרח יודעים להחזיר תשובה בפורמט שקל לבדוק. לכן הרבה benchmarks מנוסחים multiple-choice או עם extraction פשוט. פורמט שונה יכול לשנות תוצאה בלי שהיכולת השתנתה.

דוגמה: אותה שאלה במתמטיקה יכולה לקבל ציון שונה אם דורשים "A/B/C/D" לעומת הסבר חופשי.

7. Scaling laws, Chinchilla cost-1 כולל inference

Scaling laws הם חוקים אמפיריים שמקשרים בין loss לבין מספר פרמטרים, כמות דאטה compute-1. הם לא "משפט מתמטי" אלא כלי תכנון. במצגת הודגש ש-Kaplan ו-Chinchilla נתנו מסקנות שונות לגבי tradeoff בין גודל מודל לכמות טוקנים.

Chinchilla Scaling laws

$D \approx 20N$ (~20 tokens for each param)

70B parameters $\rightarrow$ 1.4T tokens

Chinchilla: כלל אצבע $D \approx 20N$, למשל 70B פרמטרים סביב 1.4T טוקנים (עמוד 378)

$C \approx 6 \cdot N \cdot D$ (rough training FLOPs)

Chinchilla rule of thumb: $D \approx 20N$ tokens

שאלה: מה היתה הטעות/העדכון ביחס ל-Kaplan?

תשובה: התוצאות המאוחרות יותר הראו שמודלים קודמים היו לעיתים under-trained: גדולים מדי ביחס לכמות הטוקנים. תחת budget קבוע, עדיף לעיתים מודל קטן יותר שמאומן על יותר טוקנים.

דוגמה: במקום 175B על מעט טוקנים יחסית, ייתכן ש-70B על הרבה יותר טוקנים ייתן loss דומה ויהיה זול יותר באינפרנס.

שאלה: למה צריך לחשב גם inference ולא רק training?

תשובה: כי מודל שמאומן פעם אחת מוגש למיליוני/מיליארדי בקשות. מודל קטן יותר שאומן על יותר דאטה יכול לעלות יותר באימון אבל לחסוך הרבה באינפרנס, במיוחד ב-decode שבו utilization נמוך.

דוגמה: חברה שמשרתת ציט המוני עשויה "לשלם" יותר באימון כדי לקבל מודל קטן ומהיר יותר בזמן שימוש.

-Post-training: SFT, RLHF, Bradley .8

Terry, PPO, DPO

Pretraining מלמד מודל להשלים טקסט. Post-training משנה את ההתנהגות כך שהמודל ימלא הוראות, ינהל דיאלוג, יהיה helpful/honest/harmless, וישתמש בפורמטים וכלים. השלב כולל SFT, preference tuning, RLHF/DPO ולעיתים RLAI/RLVR.

Reinforcement Learning from Human Feedback

- Collect human preferences over responses
- Train a reward model (prompt+response → score)
- Use reward model improve response generation
 - generate response from prompt
 - get score from reward model
 - update weights to improve scores (somehow)

RLHF: איסוף העדפות, reward model, ושיפור מדיניות (עמוד 516)

What does post-training do?

- What is learned in pre-training (base model)
- What is learned in instruct fine tuning?
- What is learned in preference tuning / RL?
- What is the *purpose* of each stage?

שאלה מרכזית: מה כל שלב ב-post-training לומד? (עמוד 544)

SFT / Instruct tuning 8.1

ב-SFT אוספים דוגמאות של instruction + response ומאמנים ב-next-token prediction על התשובה. זה מלמד פורמט, סגנון, מילוי הוראות ודיאלוג.

$$L_{\text{SFT}}(\theta) = - \sum_{(x,y)} \sum_t \log \pi_{\theta}(y_t | x, y_{<t})$$

שאלה: מה SFT לומד?

תשובה: בעיקר איך נראית תשובה טובה להוראה: סגנון, מבנה, פורמט, ציות להוראה, דיאלוג. הוא לא בהכרח מוסיף ידע עובדתי רב כי רוב הידע כבר ב-pretraining.

דוגמה: prompt: "תרגם לעברית: response: why did the chicken..." חצתה...".
למה התרנגולת

שאלה: למה SFT יכול "ללמד לשקר" או להגדיל hallucinations?

תשובה: אם דוגמאות SFT תמיד נותנות תשובה גם כשאין מספיק מידע, המודל לומד שצריך לענות בביטחון במקום לומר "לא יודע". לכן דרושים דוגמאות refusal/uncertainty ואימון honesty.

דוגמה: שאלה: "מה שם הכלב של אדם לא ידוע?" אם כל דוגמאות ההדרכה מכילות תשובות, המודל ימציא.

Preference data 8.2

ב-RLHF בני אדם משווים שתי תשובות לאותו prompt. זה זול יותר מכתבת תשובה מושלמת, ומאפשר ללמוד העדפות סובייקטיביות כמו מועילות, בהירות, בטיחות וסגנון.

שאלה: מה אם שתי התשובות טובות או שתיהן גרועות?

תשובה: אם שתיהן טובות, ההעדפה רועשת ועלולה ללמד הבדלים קטנים בסגנון. אם שתיהן גרועות, בחירת "פחות גרועה" לא אומרת מהי תשובה טובה. לכן חשוב לדגום תשובות מגוונות, לאפשר ties/ratings, ולהשתמש בהנחיות אנוטציה ברורות.

דוגמה: אם שתי תשובות מתמטיקה שגויות אבל אחת מנוסחת יפה יותר, reward model עלול ללמוד להעדיף ניסוח יפה על נכונות.

שאלה: מאיפה מגיעות שתי התשובות להשוואה?

תשובה: אפשר לקחת תשובות ממודל SFT, ממודלים שונים, ממדיניות בזמן אימון, או מתשובות שנוצרו בכוונה עם טמפרטורות שונות. מקור התשובות קובע איזה אזורים במרחב ההתנהגות reward model ילמד לדרג.

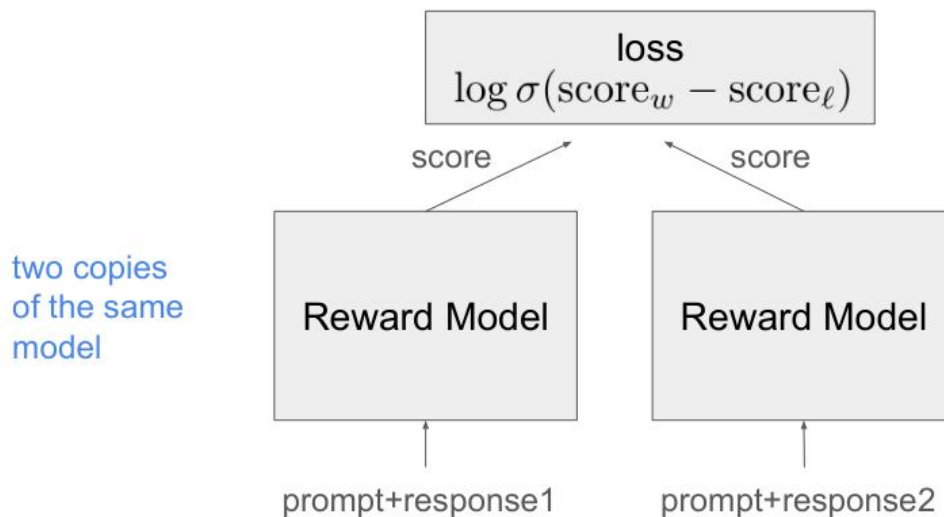
דוגמה: אם משווים רק תשובות קצרות של אותו מודל, reward model לא ילמד להעריך תשובות ארוכות עם הנמקה.

Bradley-Terry Reward Model 8.3 - הרחבה מלאה

זהו החלק שביקשת להרחיב. יש לנו נתוני pairwise preference: עבור x prompt, תשובה y_w נבחרה כטובה יותר מתשובה y_l . אבל בזמן RL אנו צריכים reward יחיד לכל תשובה. לכן

מאמנים $r_\phi(x,y)$ reward model שמחזיר ציון סקלרי. Bradley-Terry הוא מודל הסתברותי שמתרגם הפרש ציונים להסתברות שתשובה אחת תנצח את השנייה.

Train a reward model (prompt+response $\rightarrow$ score)



Reward model: שתי תשובות עוברות באותו scorer, וה- loss נבנה מהפרש הציונים (עמוד 524)

$$P_\phi(y_w > y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

$$L_{RM}(\phi) = - E_{\{(x, y_w, y_l)\}} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

$$\sigma(z) = 1 / (1 + \exp(-z))$$

אינטואיציה: אם הציון של התשובה הטובה גבוה בהרבה מהציון של התשובה הגרועה, ההסתברות קרובה ל-1 וה- loss קטן. אם ההפרש הפוך, ה- loss גדול ודוחף את המודל להגדיל את $r(y_w)$ ו- $r(y_l)$.

חישוב מספרי של Bradley-Terry

נניח $r(\text{good})=2.0$ ו- $r(\text{bad})=0.5$. ההפרש הוא 1.5 ולכן $\sigma(1.5)=0.817$. ה- loss הוא $\log(0.817)=0.202$. אם להפך $r(\text{good})=0.5$ ו- $r(\text{bad})=2.0$, אז $\sigma(-1.5)=0.182$ וה- loss הוא 1.70 - עונש גדול.

שאלה: למה לא לאמן reward model ישירות על דירוג 1-5?

תשובה: אפשר, אבל pairwise comparison לעיתים יציב וזול יותר: לא צריך לכייל בין אנוטטורים מהו "4", רק לבחור מי עדיף. Bradley-Terry מתאים בדיוק למידע יחסי כזה.

דוגמה: שני אנוטטורים יכולים לא להסכים אם תשובה היא 4 או 5, אבל להסכים שהיא טובה יותר מתשובה אחרת.

שאלה: מה החסרונות של reward model?

תשובה: הוא approximation להעדפות, עלול ללמוד קיצורי דרך, רגיש לדאטה, וניתן ל-reward hacking: המודל המייצר מוצא דרכים לקבל ציון גבוה בלי להיות באמת טוב. לכן משתמשים ב-KL ל-reference, בדיקות אנושיות ו-safety evals.

דוגמה: Reward model יכול להעדיף תשובות ארוכות ומנומסות גם כשהן לא נכונות, אם בדאטה תשובות כאלה נבחרו הרבה.

PPO-1 RLHF objective 8.4

אחרי שאימנו reward model, משפרים את π_θ policy כך שתייצר תשובות עם reward גבוה, אבל לא תתרחק מדי ממודל הייחוס/SFT. התרחקות גדולה עלולה לשבור יכולות שפה, לגרום ל-reward hacking או לשנות סגנון באופן לא רצוי.

Use reward model improve response generation

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} [r_{\theta}(x,y) - \beta \log(\pi_{\phi}^{\text{RL}}(y|x)/\pi^{\text{SFT}}(y|x))] + \gamma E_{x \sim D_{\text{pretrain}}} [\log(\pi_{\phi}^{\text{RL}}(x))]$$

How to optimize this objective?
RL!

Basic skeleton:

- sample K responses
- score them with the objective
- update towards good, away from bad

שלב RLHF: sample K responses, score, update toward good away from bad (עמוד 533)

$$\max_{\theta} E_{y \sim \pi_{\theta}(\cdot|x)} [r_{\phi}(x,y)] - \beta \text{KL}(\pi_{\theta}(\cdot|x) || \pi_{\text{ref}}(\cdot|x))$$

PPO הוא אלגוריתם RL שמעדכן מדיניות בזהירות באמצעות clipped objective. בקורס לא נדרש לרדת לכל פרטי PPO, אך כן להבין למה צריך RL: כי הטקסט הוא רצף החלטות, reward מגיע בסוף/לאחר יצירת תשובה, וצריך לעדכן הסתברויות של טוקנים שתרמו לתשובה טובה.

שאלה: למה מוסיפים KL ל-reference model?

תשובה: בלי KL, המודל עלול לנצל חולשות של reward model ולסטות משפה טבעית. KL שומר אותו קרוב למודל SFT/ייחוס, כמו רצועת ביטחון.

דוגמה: אם reward model אוהב מאוד "תודה רבה!!!", policy עלולה להוסיף זאת לכל תשובה. KL מגביל שינוי כזה.

DPO - Direct Preference Optimization 8.5

DPO מנסה להשתמש ב-preference data ישירות, בלי לאמן reward model נפרד ובלי להריץ RL מפורש. הרעיון: תחת objective עם reward, KL, אופטימלי קשור ללוג-יחס בין המודל הנלמד לבין מודל הייחוס.

Direct Preferences Optimization (DPO)

trained model

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right].$$

reference model

good response

bad response

DPO: מודל מאומן, מודל reference, תשובה טובה ותשובה רעה (עמוד 540)

RL vs. DPO

- DPO is much simpler
- DPO will stay closer to initial responses than RL
 - why?
 - is this good or bad?

שאלת מרצה: DPO פשוט יותר ונשאר קרוב יותר ל-initial responses - למה זה טוב/רע? (עמוד 542)

$$r_{\theta}(x, y) = \beta \log \left(\frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right) + \text{const}(x)$$

$$L_{\text{DPO}}(\theta) = - \mathbb{E} \log \sigma \left(\beta \left[\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x) - \log \pi_{\text{ref}}(y_w|x) + \log \pi_{\text{ref}}(y_l|x) \right] \right)$$

שאלה: למה DPO נשאר קרוב יותר ל-reference?

תשובה: הנוסחה עצמה משווה את log-prob של המודל ללוג-prob של reference. אין שלב חופשי שבו policy רודפת אחרי reward model ויכולה להתרחק הרבה. לכן DPO יציב ופשוט יותר, אך עשוי להיות פחות אגרסיבי בשינוי התנהגות.

דוגמה: אם reference כמעט לא יודע להשתמש בכלי, DPO על preference data יכול לשפר בחירה בין תשובות, אבל יתקשה להמציא יכולת חדשה לגמרי בלי דוגמאות טובות.

שאלה: DPO או RLHF - מה עדיף?

תשובה: DPO פשוט, יציב וזול יותר, מתאים לכוון העדפות על בסיס pairwise data. RLHF גמיש יותר ויכול לשלב reward מורכב או אינטראקציה, אך קשה ויקר יותר, רגיש ליציבות ול-reward hacking.

דוגמה: לשיפור סגנון/מועילות: DPO יכול להספיק. למשימות עם תגמול סביבתי, קוד שעובר tests או reasoning עם RL/RLVR verifier מתאים יותר.

RLAIF, Constitutional AI, RLVR .9

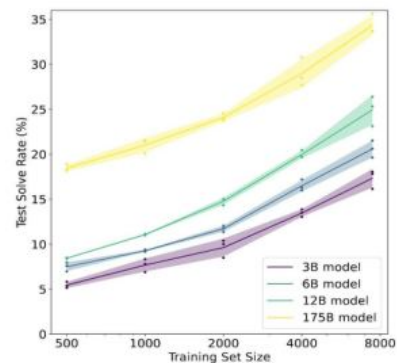
reasoning models-1

RLAIF מחליף חלק מהמשוב האנושי במשוב ממודל AI. Constitutional AI מוסיף עקרונות מפורשים שמנחים critique/revision או preference. RLVR - Reinforcement Learning with Verifiable Rewards - משתמש בבודק אוטומטי: תשובה נכונה מקבלת reward.

Fine-tuning on Reasoning Problems

- Fine-tune LMs on GSM8K (arithmetic reasoning)
- One may conjecture that, to achieve >80%, one needs **100x more training data** for 175B model
- Another option is to **increase model sizes**, which is expensive.
- Other than these, how else can we improve the model performance on tasks that require multi-step reasoning?

(Cobbe et al. 2021)



CoT: Adding “thought” before “answer”

Q: “Elon Musk”

A: the last letter of “Elon” is “n”. the last letter of “Musk” is “k”. Concatenating “n”, “k” leads to “nk”. so the output is “nk”. **thought**

Q: “Bill Gates”

A: the last letter of “Bill” is “l”. the last letter of “Gates” is “s”. Concatenating “l”, “s” leads to “ls”. so the output is “ls”.

Q: “Barack Obama”

A:

Chain-of-Thought: דוגמאות עם צעדי מחשבה לפני תשובה (עמוד 721)

Test time compute

Scaling test time compute:

- Ensembling: run multiple chains, then vote on answer
- Tree-of-thought
- Improve chains of thought quality and length

Test-time compute: ensembling, tree-of-thought ושיפור CoT (עמוד 755)

Test time compute – “Reasoning models”

- RLVR: RL with **Verifiable Rewards**
- For each training sample:
 - Sample K chains of thoughts + answers
 - a.k.a “trajectories”, “roll-outs”
 - If answer is correct, reward is 1
 - If answer is incorrect, reward is 0
 - Choose your fav RL algo (for example GRPO)

(עמוד 769) RLVR: sample K trajectories, reward 1/0 לפי נכונות

LLM Post-training Techniques recap

- SFT / Instruct tuning
- RLHF / DPO
- RLAIF
- (SFT AIF)
- RLVR

Recap של שיטות RLVR, RLAIF, DPO/RLHF, SFT: post-training (עמוד 806)

שאלה: למה CoT עוזר?

תשובה: בבעיות רב-שלביות, הפקת צעדי ביניים מאפשרת למודל לפרק את המשימה, לשמור מידע ביניים בטקסט, ולהשתמש ביותר compute בזמן inference. זה לא מבטיח שההנמקה נאמנה, במיוחד במודלים שאומנו ב-RL.

דוגמה: במקום לענות מיד "nk", המודל כותב: last letter of Elon is n; last letter of Musk is k; concatenate → nk

שאלה: מתי RLVR עובד?

תשובה: כשאפשר לייצר verifier אמין לתשובה הסופית, וכשיש סיכוי שלפחות אחת מ-K הדגימות תהיה נכונה. הוא מתאים למתמטיקה, קוד עם tests, פורמטים פורמליים, ולעיתים QA עם תשובה חד-משמעית. הוא פחות מתאים לכתיבה יצירתית או תשובות subjective.

דוגמה: ב-GSM8K אפשר לבדוק תשובה מספרית. בקוד אפשר להריץ unit tests. ב"כתוב סיכום יפה" אין verifier פשוט.

שאלה: למה reasoning traces לא תמיד נאמנים?

תשובה: במודלים שעברו RL כדי להגיע לתשובה נכונה, ה-chain-of-thought הוא פעולה שמועילה ל-reward, לא בהכרח הסבר אנושי אמיתי לתהליך. לכן לא צריך להתייחס אליו כראיה מלאה למה שהמודל "חשב".

דוגמה: מודל יכול לייצר הסבר יפה אחרי שהגיע לתשובה מתוך pattern memorized.

שאלה: איך אפשר לשלוט ב-reasoning effort?

תשובה: אפשר לנתב למודלים שונים, או להשתמש ב-conditioning tokens שמייצגים effort/length bins/think mode. בזמן SFT מלמדים את המודל לקשר token מיוחד לרמת פירוש.

דוגמה: prefix כמו <reasoning:high> גורם למודל להפיק יותר צעדי ביניים לפני התשובה.

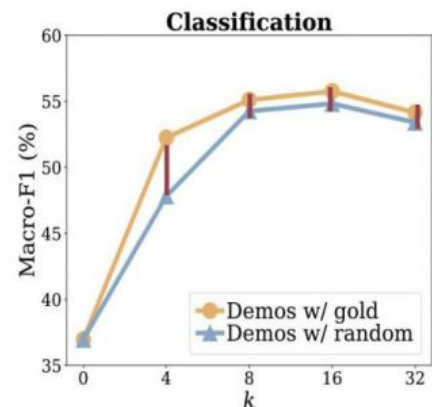
In-Context Learning, Prompting .10

biases-1

ICL הוא היכולת של מודל לבצע משימה חדשה מתוך דוגמאות בקונטקסט, בלי עדכון משקולות. זה שימושי אך רגיש מאוד לפורמט, לסדר הדוגמאות, לשמות התוויות ולדאטה האימוני.

Impact of Input-Output Mapping

- Vary number of demonstrations.
- Models see a small performance drop (0–5%) with random labels
- **Takeaway:**
 - Ground truth input-label mapping in the prompt is not as important as we thought!



מחקר שמראה שמיפוי input-label בדוגמאות פחות חשוב משחשבנו (עמוד 704)

שאלה: למה ICL עובד?

תשובה: אין תשובה אחת. הסברים אפשריים: ה-pretraining מלא במבנים מקבילים כמו רשימות וטבלאות; המודל מבצע סוג של Bayesian inference על task/concept; או מדמה במידה מסוימת עדכון פנימי. המצגת מדגישה שהפורמליזציות אינן מסבירות הכל.

דוגמה: בדוגמאות sentiment, המודל מזהה שהפורמט הוא "טקסט → label" וממשיך באותו דפוס.

שאלה: מהם majority label bias ו-recency bias?

תשובה: אם בדוגמאות יש רוב לתווית מסוימת, המודל נוטה להחזיר אותה. אם הדוגמה האחרונה מכילה label מסוים, היא יכולה להשפיע יותר. לכן סדר ובחירת הדוגמאות הם hyperparameters.

דוגמה: ארבע דוגמאות: שלוש positive ואחת negative. גם אם הטקסט החדש שלילי, המודל עשוי להטות ל-positive.

שאלה: מתי labels כן חשובים בדוגמאות?

תשובה: במשימות שבהן התווית שרירותית/חדשה והמודל לא יכול להסיק אותה מהשם, mapping נכון חשוב מאוד. למשל אם "A" פירושו סרקזם ו-"B" פירושו literal, random labels יפגעו.

דוגמה: אם label הוא "book" מול "transportation", למילים יש prior מה-pretraining; אם label הוא "X17" מול "Q9", צריך ללמוד mapping מהדוגמאות.

Code models, instruction following .11

RL-1 עם execution feedback

קוד הוא תחום מיוחד כי יש הרבה דאטה טקסטואלי, פורמטים מובנים, וחשוב במיוחד: אפשר להריץ בדיקות. לכן קוד מתחבר ל-RLVR/RLEF ול-agentic loops.

Training for coding tasks

Can rely on resources such as:

- existing code on internet (+ comments)
- books, discussion forums, stack-overflow
- coding competitions/riddles + hand written verifiers
- existing software projects with tests
 - closed issues/PRs + commits + tests
- (generated) descriptions + (generated) tests
- "recycled" data: use model to rewrite current code with constraints.
"implement this code without using numpy". see that all tests pass.
Add this to training data.

מקורות דאטה לאימון קוד: אינטרנט, פורומים, תחרויות, פרויקטים עם tests, דאטה ממוחזר (עמוד 821)

Training for coding tasks

RL:

- Like RLVR (via tests) but:
 - hard: what if all produced code is bad?
 - richer feedback: not just 0/1 but “# of passed tests”
- Can use additional signals:
 - **RLEF** → RL with **Execution Feedback**

RL לקוד: דומה ל-RLVR אך feedback יכול להיות מספר tests שעברו (עמוד 822)

שאלה: מהן משימות קוד לפי התקדמות?

תשובה: השלמת שורה/פונקציה, יצירת קוד מהוראה, Fill-in-the-Middle, שינוי קוד, תיקון באגים, עבודה עם multiple files, ולבסוף agent שכותב-מריץ-מתקן.

דוגמה: FIM: נותנים prefix ו-suffix של קובץ ומבקשים למלא את האמצע.

שאלה: מה מיוחד באימון קוד עם tests?

תשובה: אפשר לקבל reward אוטומטי: כמה tests עברו. זה מאפשר RL/RLEF, אך יש בעיות: tests חלקיים, overfitting לטסטים, קוד שנראה נכון אבל לא כללי, ועלויות הרצה.

דוגמה: אם 7 מתוך 10 tests עברו, זה richer feedback מ-0/1. אבל ייתכן שהשלושה החסרים הם מקרי קצה חשובים.

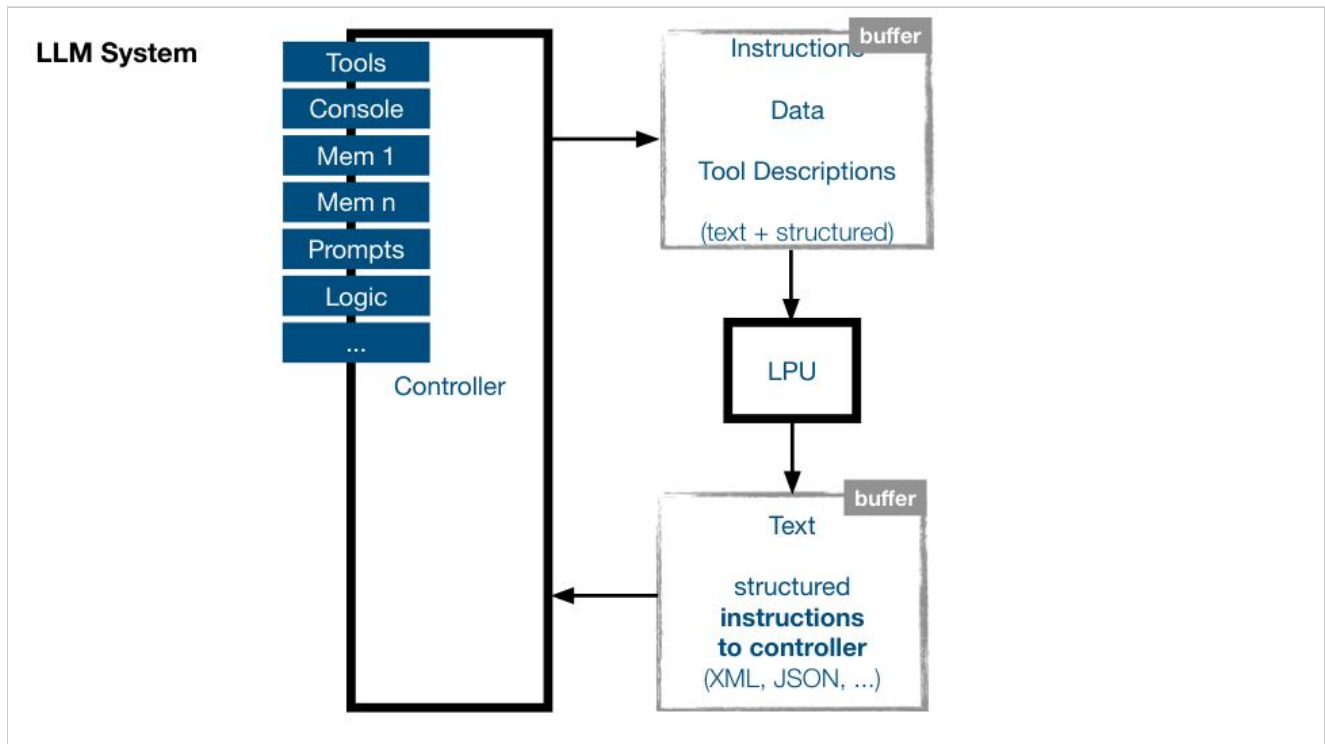
שאלת מרצה: איך לאמן instruction-following לדברים רבים בו-זמנית?

תשובה: צריך דאטה עם הוראות מורכבות הכוללות סעיפים, constraints פורמליים ובלתי פורמליים, דוגמאות של הצלחה וכישלון, ואולי verifiers חלקיים. אפשר לשלב SFT על דוגמאות, preference על איכות הציות, ו-RL כשיש בדיקות אוטומטיות.

דוגמה: הוראה: "ענה בעברית, בפחות מ-80 מילים, אל תשתמש בבולטים, כלול נוסחה".
verifier יכול לבדוק שפה/אורך/בולטים, אבל לא בהכרח איכות התוכן.

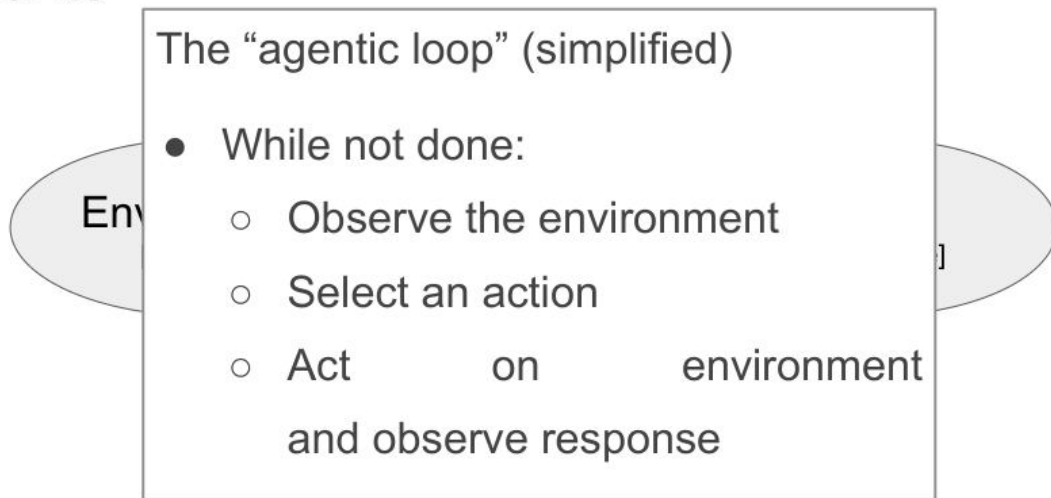
12. Agents-1 Tool calling

Tool calling הוא האינטראקציה בין LLM לבין סביבת הריצה: המודל מפיק בקשה מובנית, הקונטרולר מפרש ומריץ כלי, ואז מחזיר את התוצאה למודל. Agent הוא מערכת שחיה בסביבה, צופה, בוחרת פעולה, פועלת, וחוזרת בלולאה.



מערכת LLM עם controller, tools, memories ו-logic מסביב ל-LPU (עמוד 855)

Agents



(עמוד 871) Agentic loop: observe → select action → act → observe response

Common pattern: ReAct agent

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

¹Department of Computer Science, Princeton University

²Google Research, Brain team

¹{shunyuy, karthikn}@princeton.edu

²{jeffreyzhao, dianyu, dunan, izhak, yuancao}@google.com

ReAct: reasoning step לפני בחירת פעולה (עמוד 917)

שאלה: מה ההבדל בין tool לבין action?

תשובה: Tool הוא מקרה של action שהמערכת יודעת להריץ. בסביבת ציאט tool יכול להיות search/calculator; בסביבת רובוט action הוא תנועה; בסביבת קוד action הוא עריכת קובץ או הרצת tests. ההבחנה תלויה בסביבה.

דוגמה: web_search(query) הוא tool; בדפדפן, click(x,y) הוא action; בטרמינל, run_command(cmd) הוא action מסוכן יותר.

שאלה: מהם single-step, static multi-step, dynamic multi-step?

תשובה: single-step: בחירת כלי אחת. static multi-step: תוכנית ידועה מראש, DAG של קריאות. dynamic multi-step: הצעד הבא תלוי בתוצאה הקודמת. זה קשה יותר כי צריך התאוששות מטעויות ותכנון.

דוגמה: שאלה "מה מזג האוויר?" היא single-step. "מצא טיסה, בדוק מלון, השווה מחיר, הזמן" הוא dynamic multi-step.

שאלה: איך מאמנים tool calling?

תשובה: SFT על הדגמות קריאות כלי, synthetic data מתוך API descriptions, דאטה אנושי, דאטה ממודל חזק, ו-RL אם אפשר לבנות סביבות/verifiers. חשוב להחליט האם לכלול tool outputs ב-loss או רק בהיסטוריה.

דוגמה: המודל רואה: Tool → JSON tool call → Assistant emits → User asks weather → returns 27C → Assistant answers naturally.

שאלה: למה הכנסה של tool output ל-loss עשויה לעזור, ומה הסיכון?

תשובה: היא מלמדת את המודל להבין ולשחזר מבנה של tool outputs, אבל עלולה לגרום לו להמציא tool outputs למשתמש במקום לבצע כלי. לכן המערכת צריכה להסתיר raw tool outputs ולכפות פורמט/roles.

דוגמה: אם המודל למד ש-output של get_weather נראה כמו JSON, הוא עלול לכתוב JSON מזויף בלי קריאה אמיתית.

.13 Inference efficiency: latency, throughput, KV-cache, batching

אינפרנס ב-LLM שונה מאימון. אין backprop, אבל יש שירות בקשות בזמן אמת, זיכרון מוגבל, ו-decode רציף טוקן אחר טוקן. לכן bottleneck מרכזי הוא לעיתים memory bandwidth ולא FLOPs.

Efficiency metrics

- Cost (\$)
- Latency
 - Time to first token
 - Time to output token
- Throughput

מדדי יעילות: cost, latency, time-to-first-token, time-per-output-token, throughput (עמוד 953)

KV cache

- Reminder:
 - attention works using K,Q and V vectors
 - position q attends to previous positions by their K values
 - then, previous V values are aggregated into a new value
 - V is fed to MLP. K is output of MLP.
 - Once K and V values are computed, they don't change.
- **K and V values can be cached for future sequence positions.**

KV cache: K,V של טוקנים קודמים אינם משתנים ולכן אפשר לשמור אותם (עמוד 985)

Batching

- What does batching do?
 - to latency?
 - to throughput?
- Can you relate batching to memory transfer and FLOPs?

Batching: השפעה על latency ו-throughput (עמוד 998)

Speculative Decoding

Decoding is slow because we only generate one token at a time

→ attempt to decode more tokens at a time

Use a cheap model to generate next k tokens.

Feed all k at the same time to expensive model.

Retain the $k-m$ that matched the cheap model guess.

Speculative decoding: מודל זול מציע כמה טוקנים, מודל יקר מאמת (עמוד 1028)

FlashAttention

Faster attention computation by smarter GPU programming

- still n^2 compute
- but better memory access
- and smaller memory footprint ($n^2 \rightarrow n$)

FLASHATTENTION: Fast and Memory-Efficient Exact Attention
with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[‡], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University

[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY
{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu,
chrismre@cs.stanford.edu

June 24, 2022

FlashAttention: עדיין $O(n^2)$ compute אבל גישה חכמה יותר לזיכרון (עמוד 1033)

Store fewer KV's

- Main idea: in multi-head attention, share Keys, Values

multiple heads

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = W^Q \mathbf{h}_t,$$

$$[\mathbf{k}_{t,1}; \mathbf{k}_{t,2}; \dots; \mathbf{k}_{t,n_h}] = \mathbf{k}_t = W^K \mathbf{h}_t,$$

$$[\mathbf{v}_{t,1}; \mathbf{v}_{t,2}; \dots; \mathbf{v}_{t,n_h}] = \mathbf{v}_t = W^V \mathbf{h}_t,$$

MQA/GQA: שיתוף K,V בין ראשים להקטנת KV cache (עמוד 1043)

$$\text{KV per token} \approx 2 \cdot \text{num_heads} \cdot \text{head_dim} \cdot \text{bytes} \cdot \text{layers}$$

שאלה: מה ההבדל בין latency ו-throughput?

תשובה: Latency הוא זמן למשתמש בודד: time to first token וזמן לטוקן הבא. Throughput הוא כמה טוקנים/בקשות המערכת משרתת ביחידת זמן. batching משפר throughput אך יכול להוסיף latency כי בקשה ממתינה לבאצ'.

דוגמה: שרת יכול לחכות 20ms כדי לאסוף עוד בקשות לבאצ'. זה משפר ניצולת GPU אבל המשתמש הראשון מחכה יותר.

שאלה: למה decode memory-bound ו-prefill compute-bound?

תשובה: ב-prefill מעבדים את כל הפרומפט במקביל ויש הרבה matrix multiplications. ב-decode מייצרים טוקן אחד בכל פעם, אבל צריך לקרוא את כל KV cache של ההקשר. מעט חישוב יחסית להרבה קריאות זיכרון.

דוגמה: בטוקן אחרי 40k context, צריך לגעת ב-KV של 40k טוקנים - הרבה זיכרון בשביל טוקן אחד.

שאלה: איך quantization עוזרת?

תשובה: פחות bytes לפרמטר/אקטיבציה מקטינים תעבורת זיכרון, מאפשרים להכניס מודל/קונטקסט גדול יותר ל-GPU, ולעיתים משתמשים בפורמטים מהירים יותר. אבל היא עלולה לפגוע באיכות ודורשת כיוול.

דוגמה: BF16 ל-INT8 חותך בערך פי 2 בזיכרון משקולות; INT4 אפילו יותר, אך מסוכן יותר לאיכות.

שאלה: מתי speculative decoding נכשל?

תשובה: אם המודל הקטן מציע טוקנים שהמודל הגדול לא מאשר, לא חוסכים הרבה. השיטה עובדת כשהמודל הזול מנבא טוב את המודל היקר, במיוחד באזורים קלים/דטרמיניסטיים.

דוגמה: בהשלמת boilerplate קוד, מודל קטן עשוי לנחש היטב; בשאלה קשה יצירתית, פחות.

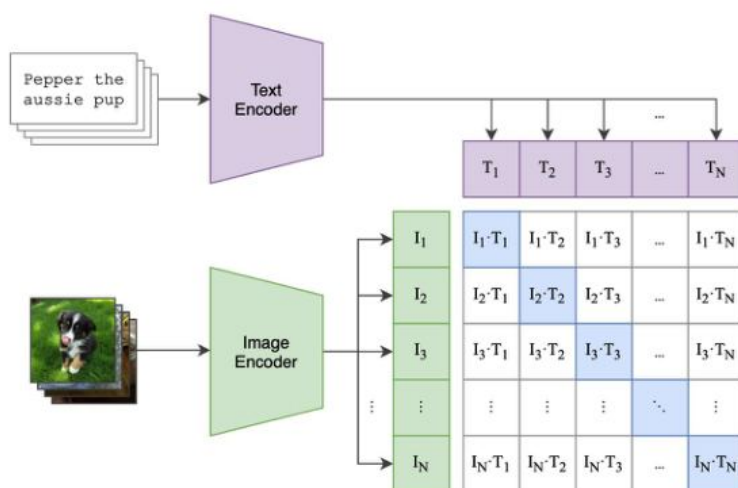
.14 Vision-Language Models: CLIP

SigLIP, ViT, LLaVA

VLM מחבר ייצוגים חזותיים למודל שפה. הקורס מדגיש שהשפה דיסקרטית והראייה רציפה, ולכן צריך גשר בין modalities: vision encoder, adapter ו-LLM.

CLIP – Training Objective

(1) Contrastive pre-training



from CLIP paper

CLIP training objective: התאמת תמונות וטקסטים במרחב משותף (עמוד 1107)

CLIP → SigLIP

Sigmoid

- ~~Contrastive~~ Language Image Pretraining

$$-\frac{1}{2|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \left(\overbrace{\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|} e^{t\mathbf{x}_i \cdot \mathbf{y}_j}}}^{\text{image} \rightarrow \text{text softmax}} + \overbrace{\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|} e^{t\mathbf{x}_j \cdot \mathbf{y}_i}}}^{\text{text} \rightarrow \text{image softmax}} \right) \quad \text{CLIP}$$

SigLIP

$$-\frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{j=1}^{|\mathcal{B}|} \log \underbrace{\frac{1}{1 + e^{z_{ij}(-t\mathbf{x}_i \cdot \mathbf{y}_j + b)}}}_{\mathcal{L}_{ij}}$$

where z_{ij} is the label for a given image and text input, which equals 1 if they are paired and -1 otherwise. At initial-

CLIP מול SigLIP: שינוי ה-objective ל-sigmoid pairwise (עמוד 1135)

The LLaVA approach

Also used in Qwen and many others

(Large Language and Vision Assistant)

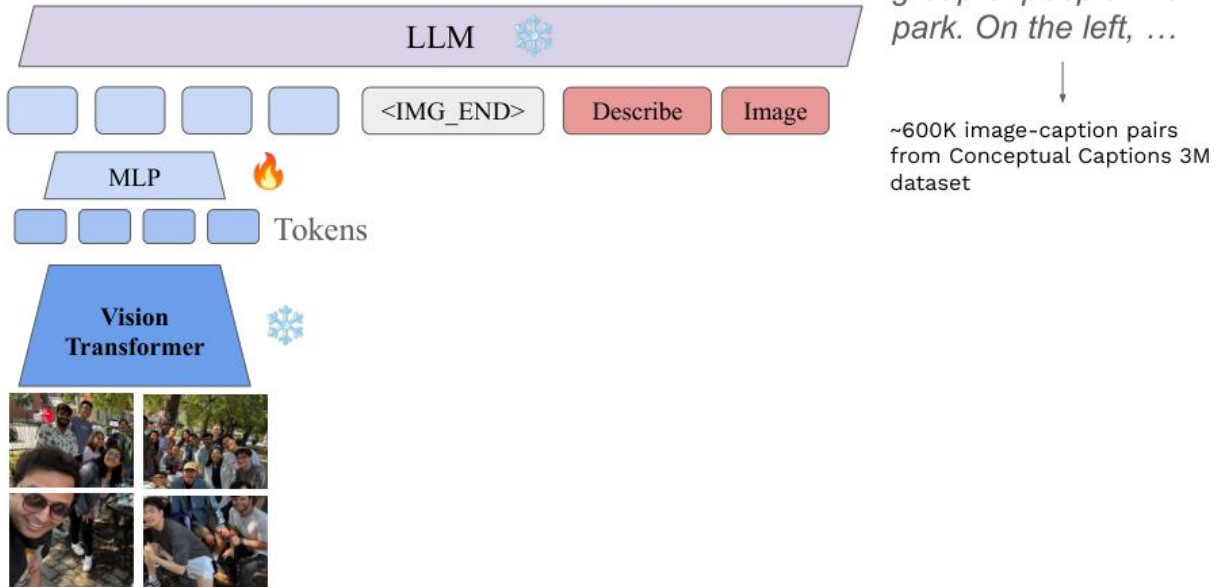
Visual Instruction Tuning

Haotian Liu^{1*}, Chunyuan Li^{2*}, Qingyang Wu³, Yong Jae Lee¹

¹University of Wisconsin–Madison ²Microsoft Research ³Columbia University
<https://llava-vl.github.io>

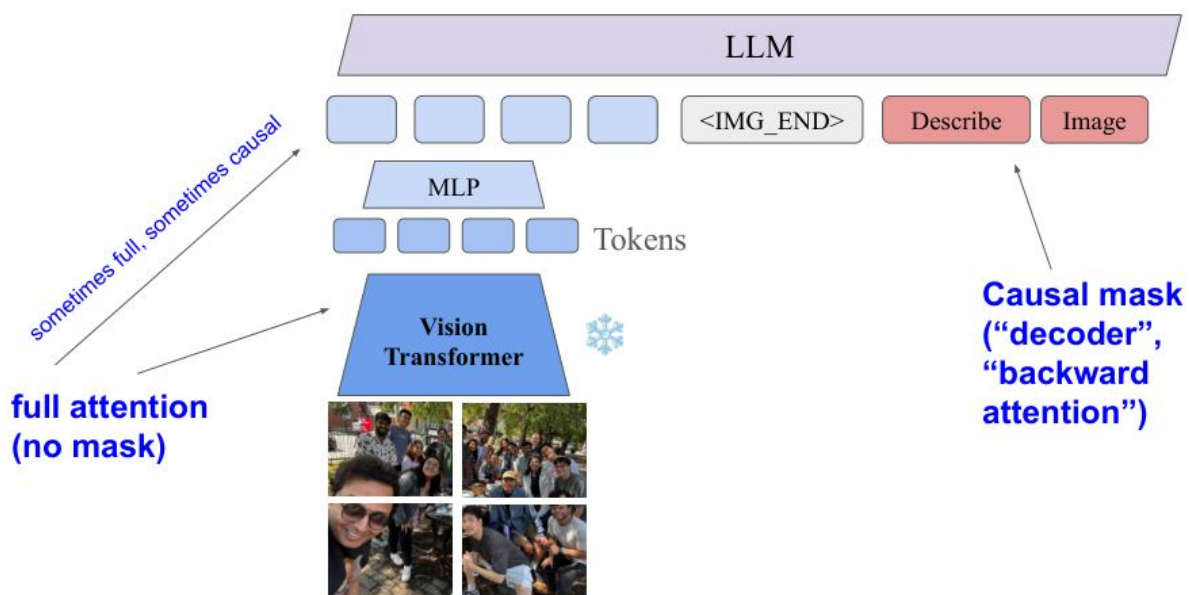
LLaVA-style: vision encoder + MLP adapter + LLM (עמוד 1178)

Training: Stage 1



שלב 1 באימון VLM: image-caption pairs, רק adapter מאומן (עמוד 1190)

Masking differences



הבדלי masking: causal mask ב-LLM מול full attention ב-vision encoder (עמוד 1196)

Unimodal baselines are still surprisingly effective

Take a VQA benchmark and try:

- Question only baseline (no image)
- Image only baseline (no question)

Often result in surprisingly high accuracy

(Why?)

VQA baselines: question-only/image-only יכולים להצליח באופן מפתיע (עמוד 1238)

```
CLIP: maximize similarity(image_i, text_i), minimize similarity(image_i, text_j), j≠i
```

שאלה: מה CLIP עושה ומה הוא לא עושה?

תשובה: CLIP לומד embeddings משותפים לתמונה וטקסט ומתאים לחיפוש/סיווג zero-shot. הוא לא מודל גנרטיבי של שיחה ולא יודע להפיק תשובה ארוכה כמו LLM.

דוגמה: אפשר לשאול איזה caption מתאים לתמונה, אבל לא לנהל דיאלוג מורכב על התמונה.

שאלה: למה SigLIP?

תשובה: במקום softmax contrastive שמחייב השוואה לכל הזוגות בבאצ', SigLIP מדרג כל זוג image-text באופן עצמאי עם sigmoid. זה יכול לשפר scaling/efficiency בבאצ'יים גדולים.

דוגמה: ב-CLIP כל תמונה מתחרה בכל הטקסטים בבאצ'; ב-SigLIP כל זוג מקבל label מתאים/לא מתאים.

שאלה: איך LLaVA מחבר תמונה ל-LLM?

תשובה: Vision encoder הופך תמונה ל-adapter/MLP, tokens/embeddings ממפה אותם למרחב של ה-LLM, ואז הם מוזנים כטוקנים לפני הטקסט. תחילה מאמנים adapter על captioning, אחר כך post-training על chat/QA מולטימודלי.

דוגמה: תמונה → <image → prompt: <ViT patches → MLP → "visual tokens" → Describe image → LLM מייצר תשובה.

שאלה: למה question-only baseline ב-VQA מצליח?

תשובה: כי datasets מכילים biases: סוגי שאלות מסוימים קשורים לתשובות נפוצות. למשל "What color is the sky?" לרוב blue. לכן מודל יכול לנחש בלי תמונה. זה מלמד שצריך baselines ולבדוק האם המודל באמת משתמש במודאליות.

דוגמה: אם רוב השאלות "How many wheels?" בתמונות רכב עונות 4, question-only ילמד לנחש 4.

Interpretability / Explainability .15

ודגשי מבחן

בשיעור 12 המרצה עבר גם על המבחן וגם על interpretability. הרעיון המרכזי: לא מספיק לראות קלט ופלט; רוצים להבין מה שמור באקטיבציות, מה שמור במשקולות, ואיך החישוב מתבצע.

Understanding computation through activations

Method families:

- Projections
- Clustering / Neighbours analysis
- Probing
- Interventions
 - Nulling, Noising, Erasing, Patching

משפחות שיטות: projections, clustering, probing, interventions, patching (עמוד 1305)

Projections: Logit-lens

Method:

project the activations into the vocabulary space of the network

In practice:

multiply the activations at layer L by the unembedding matrix.

pros and cons of approach?

Logit lens: הקרנת אקטיביציות למרחב vocabulary (עמוד 1309)

Projections: Sparse auto-encoders (SAE)

input vector $\mathbf{x} \in \{\mathbf{x}_i\}$, our network produces the output $\hat{\mathbf{x}}$, given by

$$\mathbf{c} = \text{ReLU}(M\mathbf{x} + \mathbf{b}) \quad (1)$$

$$\hat{\mathbf{x}} = M^T \mathbf{c} \quad (2)$$

$$= \sum_{i=0}^{d_{\text{hid}}-1} c_i \mathbf{f}_i \quad (3)$$

where $M \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{hid}}}$ are our learned parameters, and M is normalised row-wise³. Our parameter matrix M is our feature dictionary, consisting of d_{hid} rows of dictionary features $\mathbf{f}_i$. The output $\hat{\mathbf{x}}$ is meant to be a reconstruction of the original vector $\mathbf{x}$, and the hidden layer $\mathbf{c}$ consists of the coefficients we use in our reconstruction of $\mathbf{x}$.

$$\text{Loss: } \mathcal{L}(\mathbf{x}) = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2}_{\text{Reconstruction loss}} + \underbrace{\alpha \|\mathbf{c}\|_1}_{\text{Sparsity loss}}$$

SPARSE AUTOENCODERS FIND HIGHLY INTERPRETABLE FEATURES IN LANGUAGE MODELS

Hoagy Cunningham^{1,2}, Aidan Ewart^{1,3}, Logan Riggs^{1,3}, Robert Huben, Lee Sharkey⁴
¹EleutherAI, ²MATS, ³Brisol AI Safety Centre, ⁴Apollo Research
[hoagy@eleutherai.com, aidan@pratt.earth, logan@smiths.org, lee@sharkey.ai]

Sparse Autoencoders: loss לשחזור עם sparse features (עמוד 1317)

Projections: Sparse auto-encoders (SAE)

To map the feature ids to human interpretable concepts:

- collect samples for which the feature is active
- ask an LLM to describe the commonality
- attempt to use the description to create another examples for which the feature will be active (optional)

מיפוי SAE features לקונספטים אנושיים (עמוד 1319)

Activation patching

- Take activations from one example
- Inject them into another example

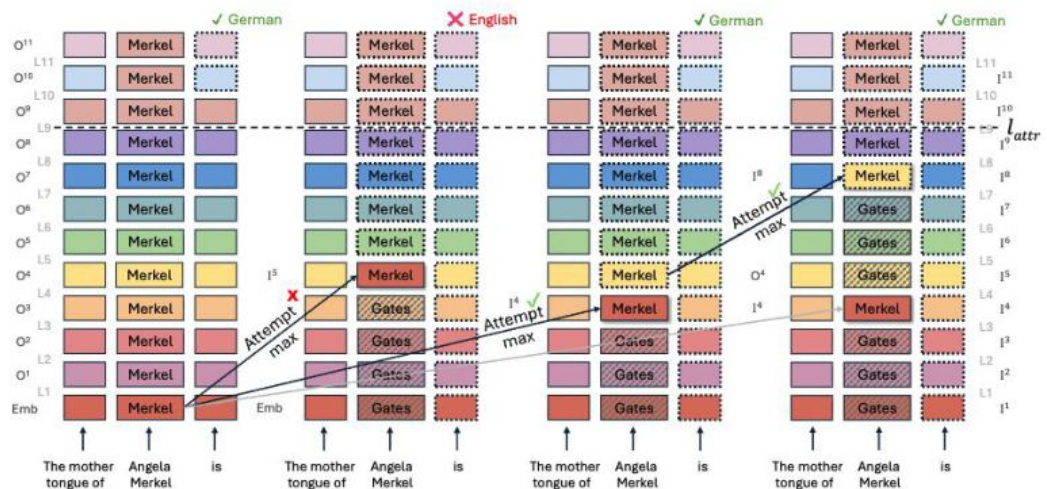
Activation patching: העברת אקטיבציות מריצה נקייה לריצה משובשת (עמוד 1335)

Activation patching

Factual Retrieval in LLMs
Is a Redundant, Distributed and Non-Contiguous Process

Counterfactual patching

patch over counterfactual prompt ("The mother tongue of Bill Gates is")



Activation patching מסמן איפה במודל נמצא המידע הרלוונטי (עמוד 1347)

Logit lens

ב-logit lens לוקחים אקטיבציה משכבה פנימית ומכפילים ב-unembedding matrix כדי לראות אילו טוקנים "מועדפים" בשלב הזה. זה נותן הצצה לאופן שבו תחזית מתפתחת לאורך שכבות.

שאלה: יתרונות וחסרונות logit lens?

תשובה: יתרון: פשוט, מהיר, נותן אינטואיציה. חסרון: שכבות פנימיות לא בהכרח נמצאות באותו מרחב כמו השכבה האחרונה; projection יכול להטעות; הוא מראה correlation ולא causal mechanism.

דוגמה: אם בשכבה 10 logit lens כבר מצביע על "Paris", זה לא מוכיח ששכבה 10 לבדה אחראית לתשובה.

Probing

ב-probing מאמנים classifier קטן על אקטיבציות כדי לנבא תכונה: למשל מספר, מין דקדוקי, אמת/שקר, או האם יש entity מסוים. אם probe מצליח, התכונה קיימת במידע האקטיבציה במובן כלשהו.

שאלה: מה הבעיה עם probing?

תשובה: Probe חזק מדי יכול ללמוד את המשימה בעצמו ולא רק לקרוא מידע קיים. גם אם מידע ניתן לקריאה, זה לא אומר שהמודל משתמש בו בפועל. לכן צריך להשוות probes פשוטים, controls, והתערבויות causal.

דוגמה: אם linear probe מזהה שפה מתוך אקטיבציה, כנראה שהמידע מקודד. אבל אם MLP גדול מזהה, אולי הוא בנה classifier מורכב בעצמו.

Sparse Autoencoders (SAE)

SAE מנסה לפרק אקטיבציות למספר קטן של features מתוך מרחב גדול מאוד. המטרה היא להתמודד עם superposition: תכונות רבות מיוצגות בשילובים של נוירונים, לא "נוירון סבתא" אחד לכל קונספט.

$$a \approx \text{Decoder}(z), \quad z = \text{Encoder}(a), \quad z \text{ sparse}$$

$$L_{\text{SAE}} = ||a - \hat{a}||^2 + \lambda ||z||_1$$

שאלה: למה אין באמת grandma neuron?

תשובה: במודלים גדולים תכונות רבות חולקות ממדים. נוירון יחיד יכול להשתתף בכמה תכונות, ותכונה אחת יכולה להתפרס על כמה נוירונים. זה יעיל יותר מבחינת ייצוג, אך קשה לפרש.

דוגמה: אותו נוירון יכול להגיב גם לצרפת וגם לבירות אירופאיות בהקשרים שונים.

שאלה: איך ממפים SAE feature לקונספט?

תשובה: אוספים דוגמאות שבהן feature פעיל, מבקשים מאדם או LLM לתאר מה משותף, ואז בודקים אם התיאור מנבא דוגמאות חדשות שמפעילות את feature. זה תהליך אמפירי ולא הוכחה מושלמת.

דוגמה: feature #18342 פעיל בדוגמאות עם כתובות דוא"ל; מתארים אותו כ-email-address feature ובודקים על דוגמאות חדשות.

Activation patching

Activation patching הוא כלי causal: מריצים input נקי שבו המודל עונה נכון, input משובש שבו הוא טועה, ואז מחליפים אקטיבציה מסוימת בריצה המשובשת באקטיבציה מהריצה הנקייה. אם הפלט מתוקן, הרכיב שהוחלף כנראה חשוב causal.

```
clean run: x_clean -> activations A_clean -> correct output
```

```
corrupted run: x_bad -> activations A_bad -> wrong output
```

```
patched run: replace A_bad[layer,position] with A_clean[layer,position]
```

שאלה: למה patching חזק יותר מ-correlation?

תשובה: כי הוא משנה רכיב ובודק השפעה על הפלט. זו התערבות causal, לא רק מדידה. עם זאת, צריך להיזהר: החלפה לא טבעית של אקטיבציות יכולה ליצור מצבים שלא מופיעים ברגיל.

דוגמה: בדוגמת "The Eiffel Tower is in Paris", אם patching של position של entity בשכבה 15 מחזיר "Paris", הרכיב הזה חשוב לשליפת העובדה.

איך להתכונן למבחן לפי שיעור 12

- ללמוד הגדרות, אבל לענות באינטגרציה: מקור התנהגות, שלב אימון, דאטה, evaluation, tradeoffs.
- להכיר datasets: מה הם מכילים, איזו יכולת הם בודקים, ומה המגבלות שלהם.
- להבין דיאגרמות: אם מציגים גרף או pipeline, להסביר מה קורה אם מזיזים נקודה או משנים רכיב.
- לא לשנן מספרים מדויקים, כן לזכור סדרי גודל: מיליונים/ביליונים, טוקנים רבים, context ארוך, KV cache גדול.
- לזהות שאלות discuss/why/consider מהשקפים ולדעת לענות על היתרונות, החסרונות והמצבים שבהם השיטה נכשלת.
- לתת תשובה שאינה סותרת את עצמה: אם אמרת ש-DPO קרוב ל-reference, אל תטען מיד שהוא יוצר יכולות חדשות חזקות בלי דאטה מתאים.

16. בנק שאלות מבחן עם תשובות מודל

שאלה: מודל מציג hallucinations רבות אחרי SFT. מהם מקורות אפשריים ומה היית עושה?

תשובה: מקורות: SFT demonstrations שעונות תמיד גם כשאין מידע; חוסר בדוגמאות refusal/uncertainty; preference data שמעדיף תשובות בטוחות; benchmark/אימון ללא בדיקת factuality. תיקון: להוסיף דאטה של "לא יודע", RAG/tool use כשצריך מידע, preference על honesty, eval factuality, ולבדוק אם הבעיה מה-base או מה-post-training.

דוגמה: שאלה על מידע רפואי לא ידוע: התשובה הרצויה היא לסייג ולהמליץ מקור מוסמך, לא להמציא אבחנה.

שאלה: חברה מציעה להגדיל batch באינפרנס כדי לשפר latency. האם זה נכון?

תשובה: לא בהכרח. Batch גדול לרוב משפר throughput וניצולת GPU, אבל עשוי להגדיל latency כי בקשות ממתיונות ו-decode צריך לסנכרן רצפים שונים. ייתכן ש-continuous batching משפר פשרה, אך לא נכון לומר שבאצי תמיד משפר latency.

דוגמה: בשירות צ'אט בזמן אמת, batch גדול מדי יגרום למשתמש הראשון לחכות.

שאלה: מתי לבחור MMLU ומתי GSM8K?

תשובה: MMLU בודק ידע רוחבי ושאלות בתחומים רבים; GSM8K בודק חשבון מילולי רב-שלבי עם תשובה ניתנת לאימות. אם רוצים reasoning אריתמטי, GSM8K מתאים יותר. אם רוצים ידע כללי/מקצועי רוחבי, MMLU.

דוגמה: מודל יכול להיות טוב ב-MMLU אך חלש בפתרון בעיות חשבון רב-שלביות.

שאלה: מה ההבדל בין reward model ל-DPO?

תשובה: Reward model לומד scorer מפורש בעזרת Bradley-Terry ואז RL משפר policy לפי הציון. DPO משתמש באותם זוגות העדפה כדי לעדכן ישירות את policy ביחס למודל reference, בלי RL מפורש ובלי reward model נפרד.

דוגמה: DPO פשוט יותר RLHF; pipeline-wise גמיש יותר אבל מסובך.

שאלה: האם CoT הוא הסבר נאמן?

תשובה: לא בהכרח. CoT יכול לעזור לביצועים ולשמש scratchpad, אבל במיוחד אחרי RL הוא אופטימיזציה להשגת תשובה, לא תיעוד אמיתי של תהליך פנימי. כדי לבדוק נאמנות צריך התערבויות/מחקר causal.

דוגמה: מודל יכול לתת תשובה ואז להצדיק אותה בדיעבד.

שאלה: איך תבדוק אם VLM משתמש בתמונה ולא רק בשאלה?

תשובה: מריצים baselines: question-only, image-only, shuffled image, counterfactual images. אם question-only מקבל דיוק גבוה, יש bias בדאטה. אפשר לבנות דוגמאות שבהן אותה שאלה על תמונות שונות דורשת תשובות שונות.

דוגמה: "What color is the car?" על תמונות עם צבעים שונים. אם המודל עונה תמיד "red", זו בעיה.

שאלה: למה MoE לא תמיד זול באינפרנס למרות שמפעילים מעט מומחים?

תשובה: כי צריך להחזיק הרבה פרמטרים בזיכרון, לבצע routing, להעביר טוקנים בין מומחים/GPU, להתמודד עם עומס לא מאוזן, ולשמור FLOPs. parallelism. לטוקן קטנים יחסית, אך memory/communication יכולים לשלוט.

דוגמה: top-2 מתוך 64 experts מפחית חישוב, אך אם המומחים על GPUs שונים יש עלות תקשורת.

שאלה: איך היית מייצר דאטה לאימון tool calling?

תשובה: מתחיל מ-API specs, יוצר סיטואציות משתמש, מייצר tool calls נכונים ותשובות אחרי tool output. מוסיף דוגמאות של שגיאות/התאוששות, פורמטים שונים, multi-step tasks. משתמש ב-SFT; אם יש סביבה/verifier אפשר גם RL.

דוגמה: API weather(city,date): יוצרים שאלה "האם אצטרך מטריה מחר בתל אביב?" → tool call → תשובה טבעית.

שאלה: מה יכול לגרום לעלייה בציון benchmark בלי שיפור אמיתי?

תשובה: Contamination, prompt overfitting, שינוי פורמט extraction, דוגמאות few-shot שנבחרו לפי test, exploit של artifacts, או מדד שאינו מייצג את היכולת שרוצים.

דוגמה: אם בוחרים demonstrations לפי הביצוע על test, זה לא הערכה הוגנת.

שאלה: איך activation patching יכול לעזור להבין factual recall?

תשובה: משווים ריצה נקייה וריצה משובשת, מחליפים אקטיבציות לפי שכבה/מיקום, ומודדים האם התשובה חוזרת לנכונה. כך ממפים איפה מידע על entity/relation זורם במודל.

דוגמה: ;clean: "Paris is capital of France"; corrupted: replace France with Italy
patch positions to see where התשובה משתנה.

17. דף נוסחאות ואלגוריתמים מרוכז

Language modeling

$$P(x_{1:T}) = \prod_t P(x_t | x_{<t})$$

$$L_{\text{NTP}} = - \sum_t \log P(x_t | x_{<t})$$

$$\text{PPL} = \exp(\text{average cross entropy})$$

Attention

$$Q=XW_Q, K=XW_K, V=XW_V$$

$$\text{Attention} = \text{softmax}((QK^T / \sqrt{d_k}) + M)V$$

$$\text{Causal mask: } M_{ij}=0 \text{ if } j \leq i \text{ else } -\infty$$

Modern blocks

$$\text{RMSNorm}(x)=g \odot x / \sqrt{\text{mean}(x^2)+\epsilon}$$

$$\text{Swish}(x)=x \cdot \text{sigmoid}(x)$$

$$\text{SwiGLU}(x)= (xW_v) \odot \text{Swish}(xW_g)$$

$$\text{RoPE: } \langle R_m q, R_n k \rangle \text{ depends on } m-n$$

$$\text{MoE: } y=\sum_{e \in \text{TopK}(x)} \text{gate}_e(x) \text{ Expert}_e(x)$$

Post-training

$$L_{\text{SFT}} = - \sum \log \pi_{\theta}(y_t \mid x, y_{<t})$$

$$\text{Bradley-Terry: } P(y_w > y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

$$L_{\text{RM}} = -E \log \sigma(r_w - r_l)$$

$$\text{RLHF: } \max E[r(x, y)] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

$$\text{DPO: } -E \log \sigma(\beta[(\log \pi_{\theta_w} - \log \pi_{\theta_l}) - (\log \pi_{\text{ref}_w} - \log \pi_{\text{ref}_l})])$$

Inference

$$\text{KV/token} \approx 2 \cdot \text{heads} \cdot \text{head_dim} \cdot \text{bytes} \cdot \text{layers}$$

$$\text{Training FLOPs rough: } C \approx 6ND$$

$$\text{Chinchilla rough optimum: } D \approx 20N \text{ tokens}$$

Interpretability

$$\text{Logit lens: } \text{logits}_L = h_L W_U$$

$$\text{SAE: } L = ||a - \hat{a}||^2 + \lambda ||z||_1$$

Activation patching: replace $A_{\text{bad}}[\text{layer}, \text{pos}]$ with $A_{\text{clean}}[\text{layer}, \text{pos}]$ and measure output change

18. Checklist אחרון לפני מבחן

- אני יודע להסביר NTP/LPU/LLM system ולתת דוגמה ללולאה.
- אני יודע לכתוב Attention עם Q,K,V ו-causal mask.
- אני יודע להסביר RMSNorm, SwiGLU/gating, RoPE, MoE ומה tradeoff של כל אחד.
- אני יודע מה זה tokenizer, BPE, PPL, BPB ולמה אין להשוות PPL בין tokenizers בלי זהירות.
- אני מכיר את datasets/benchmarks המרכזיים ומה הם בודקים.
- אני יודע להסביר benchmark contamination ודאטה filtering.
- אני יודע Bradley-Terry loss ולחשב דוגמה קטנה.
- אני יודע להשוות SFT, RLHF, DPO, RLAIIF, RLVR.
- אני יודע מתי RLVR עובד ומתי לא.
- אני יודע להסביר ICL biases: majority, recency, common token, label mapping.
- אני יודע להסביר tool calling ו-agentic loop.
- אני יודע להסביר prefill/decode, KV-cache, batching, quantization, speculative decoding.
- אני יודע להסביר VQA baselines ו-CLIP/SigLIP, LLaVA-style VLM.
- אני יודע להסביר activation patching ו-logit lens, probing, SAE.
- בכל תשובה אני מציין מקור התנהגות אפשרי, שלב אימון, דאטה מתאים, eval מתאים ו-tradeoffs.

מקורות פנימיים: תמלילי TXT מתוך LLM_TRANSC.zip והמצגת המאוחדת all_lectures.pdf. השקפים הוויזואליים בחוברת הם עיבודים/תמונות ממסמך המצגות שסופק.